Empirical Evaluation of Exact and Approximation Algorithms for the 0/1 Knapsack Problem

Muhammad Sultan Haidar Cheema

December 16, 2025

Contents

1 Introduction **2**

2 Algorithms Considered **2**
2.1 Brute Force 2
2.2 Greedy Heuristics 2
2.3 Dynamic Programming 3
2.4 Branch and Bound 3

3 Experimental Setup **3**
3.1 Synthetic Instances 3
3.2 OR-Library Benchmark Instances 3
3.3 Evaluation Metrics 4

4 Results on Synthetic Instances **4**
4.1 Runtime Scaling 4
4.2 Solution Quality 5
4.3 Relative Error 6
4.4 Representative Synthetic Table for $n = 25$ 6

5 Results on OR-Library Benchmark Instances **7**
5.1 Optimality Gaps 7
5.2 Runtime on Benchmarks 7
5.3 Solution Values on Benchmarks 8

6 Optimality Differences on OR-Library Instances **8**

7 Key Observations **9**

8 Conclusion **9**

9 Project Planning and Organization **10**

Abstract

The 0/1 Knapsack Problem is a fundamental NP-hard combinatorial optimization problem with numerous applications in resource allocation, logistics, and decision-making. This report presents a systematic empirical evaluation of several classical exact and heuristic algorithms, including brute force, three greedy variants, dynamic programming, and branch and bound. Experiments are conducted on both synthetic instances of increasing size and established OR-Library (MKNAP1) benchmark problems with known optimal solutions. Synthetic results show brute force becoming infeasible once n exceeds 25 items (up to about 700 s at $n = 28$), while branch and bound solves instances with up to 150 items in under 0.011 s and dynamic programming remains below 0.41 s. On OR-Library instances, branch and bound consistently matches the published optima on the corresponding single-constraint reductions, whereas greedy heuristics exhibit non-trivial optimality gaps. Overall, the study highlights that while exact methods guarantee optimality, branch and bound offers the best practical trade-off between optimality and runtime, and greedy-by-ratio remains a strong baseline for very fast approximate solutions.

1 Introduction

The 0/1 Knapsack Problem is defined as

$$\max \sum_{j=1}^{n} p_j x_j \quad \text{subject to} \quad \sum_{j=1}^{n} w_j x_j \leq C, \quad x_j \in \{0, 1\}.$$

Despite its simple formulation, the problem is NP-hard, and no polynomial-time algorithm is known for solving all instances to optimality. This has motivated a broad spectrum of algorithms ranging from exponential-time exact methods to fast heuristics with no worst-case optimality guarantees.

The objective of this report is to move beyond asymptotic complexity theory and provide a careful experimental study of algorithmic behavior in practice. The analysis focuses on how runtime and solution quality evolve with problem size, and how algorithm performance differs between randomly generated instances and structured OR-Library benchmarks with known optimal values.

2 Algorithms Considered

2.1 Brute Force

The brute force algorithm enumerates all 2^n subsets of items and returns the feasible subset with maximum total value. Its exponential complexity makes it a reference method rather than a practical solver: in the synthetic experiments, brute force runs in around 1.1×10^{-3} s at $n = 10$, increases to about 2.47 s at $n = 20$, and reaches roughly 122 s at $n = 25$, after which it is disabled. On OR-Library benchmarks, brute force is only used when the single-constraint reduction yields fewer than 30 items; for example, on Problem 3 ($n = 15$) it requires about 0.040 s, and on Problem 5 ($n = 28$) it requires roughly 698 s.

2.2 Greedy Heuristics

Three greedy heuristics were implemented:

- **Greedy by Value**: sort items by decreasing value.
- **Greedy by Weight**: sort items by increasing weight.
- **Greedy by Value-to-Weight Ratio**: sort items by density.

Each heuristic runs in $O(n \log n)$ time due to sorting. On synthetic instances, their runtimes remain on the order of 10^{-5}–10^{-4} seconds across all tested sizes; for example, at $n = 150$ the greedy methods have runtimes around $6 \cdot 10^{-5}$–$7 \cdot 10^{-5}$ seconds. They do not guarantee optimality, but the ratio-based heuristic regularly comes very close to the optimal dynamic programming value.

2.3 Dynamic Programming

Two dynamic programming (DP) variants were implemented:

- Bottom-up DP with table reconstruction;
- Top-down DP with memoization.

Both variants achieve optimal solutions with pseudo-polynomial complexity $O(nC)$ in time and memory. On synthetic instances, bottom-up DP scales from a few milliseconds at $n = 10$ to approximately 0.40 s at $n = 150$, while top-down DP increases from below 1×10^{-3} s to slightly above 1.16 s over the same range. On benchmark instances with moderate capacities, DP remains practical and matches the known optimal values.

2.4 Branch and Bound

Branch and bound explores a decision tree over items and prunes subtrees using an upper bound derived from the fractional knapsack relaxation. In the implementation used here, nodes are ordered by value-to-weight ratio and fractional bounds are recomputed to tighten pruning. Although its worst-case complexity remains exponential, branch and bound combines exactness with strong practical efficiency: in synthetic tests, it solves all instances up to $n = 150$ in under 0.011 s, and on OR-Library problems it typically stays in the $4.4 \cdot 10^{-5}$–1.6×10^{-2} s range while always attaining the benchmark optimal value under the single-constraint formulation.

3 Experimental Setup

3.1 Synthetic Instances

Synthetic instances were generated with $n \in \{10, 20, 25, 60, 100, 150\}$. Item weights and values were sampled randomly, and the knapsack capacity was set as a fixed fraction of the total weight to avoid trivial full or empty solutions. For each n, runtimes and solution values were recorded for all algorithms, with brute force restricted to $n \leq 25$ due to its cost.

3.2 OR-Library Benchmark Instances

Benchmark experiments used the OR-Library MKNAP1 dataset, which comprises multidimensional knapsack instances from project selection applications. The dataset contains seven problems with item counts and multi-constraint dimensions

$$(6, 10),\ (10, 10),\ (15, 10),\ (20, 10),\ (28, 10),\ (39, 5),\ (50, 5),$$

and published optimal objective values ranging from 3800.0 up to 16537.0. To interface with 0/1 knapsack solvers, each problem was reduced to a single-constraint instance by taking one capacity constraint at a time.

3.3 Evaluation Metrics

The following metrics were recorded for each run:

- Runtime in seconds, averaged over multiple trials when appropriate;
- Total solution value;
- Relative error with respect to a reference optimal solution (DP on synthetic instances; OR-Library optima on benchmarks).

All reported numbers are obtained directly from the experimental runs and aggregated via programmatic analysis.

4 Results on Synthetic Instances

4.1 Runtime Scaling

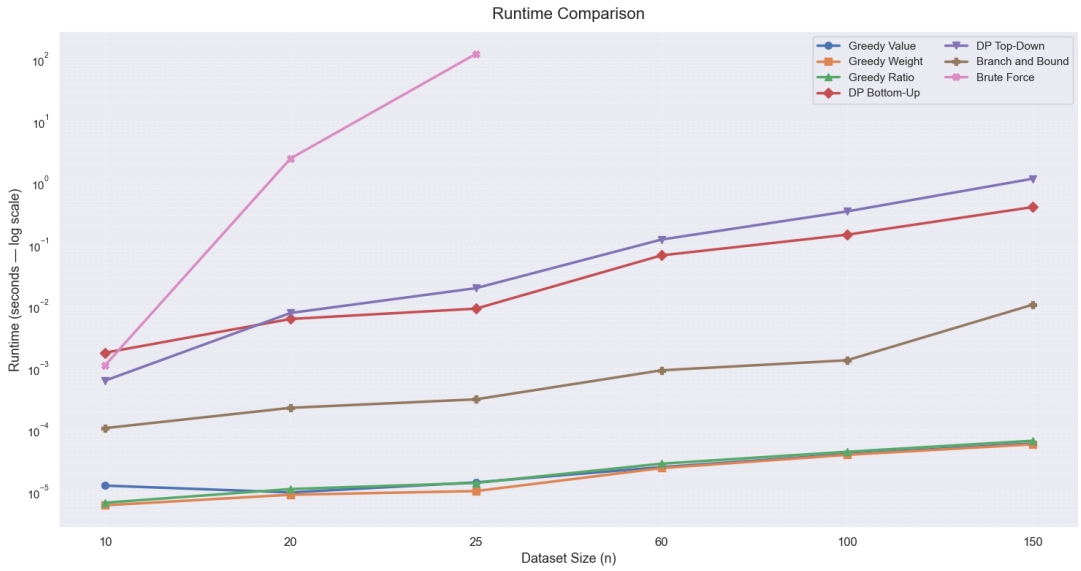


Figure 1: Runtime comparison on synthetic instances (log scale).

The runtime plot shows the expected separation between exponential and polynomial (or pseudo-polynomial) methods. Brute force exhibits exponential growth in n and becomes infeasible beyond $n = 25$, where the running time exceeds 120 s. Dynamic programming grows smoothly with both n and capacity and reaches roughly 0.40 s for bottom-up and 1.16 s for top-down at $n = 150$. Branch and bound demonstrates excellent practical performance, solving even the largest synthetic instances in under 0.011 s, while greedy heuristics remain effectively constant time on this scale, with runtimes on the order of 10^{-5} seconds.

4.2 Solution Quality

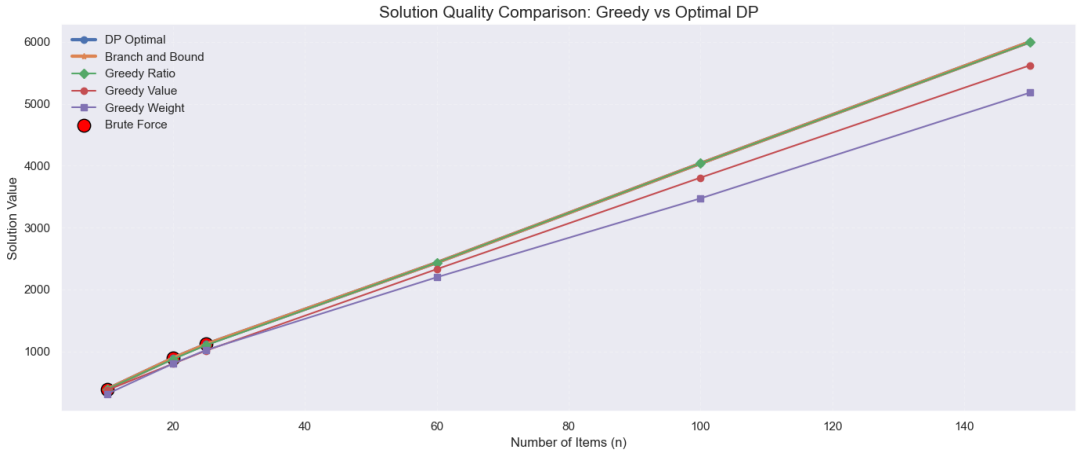


Figure 2: Solution value comparison between greedy heuristics and optimal DP.

The solution quality plot reports, for each synthetic instance size, the objective values obtained by the three greedy strategies alongside the optimal dynamic programming value. Across all n, greedy-by-ratio closely tracks the optimal curve and typically lies only a few percent below the DP value, while greedy-by-value and greedy-by-weight produce noticeably lower values and display larger fluctuations. For instance, for a representative instance with $n = 25$, DP (and branch and bound) attain a value of 1116, greedy-by-ratio reaches 1099 (98.5% of optimal), whereas greedy-by-value and greedy-by-weight yield 1012 and 1021 respectively, corresponding to roughly 9–10% loss in objective value.

As problem size increases, the absolute gap between heuristic and optimal solutions grows, but the relative ranking of the three greedy strategies remains stable: the ratio-based rule consistently dominates the value- and weight-based rules. This observed ordering aligns well with the theoretical intuition that value-to-weight density is a more informative selection criterion than value or weight alone.

4.3 Relative Error

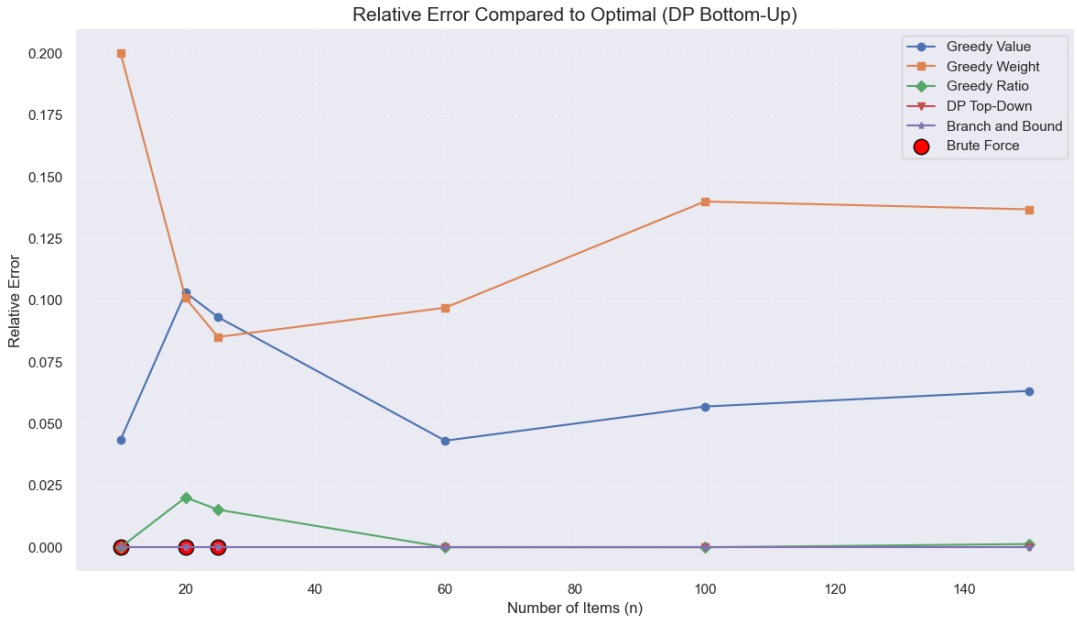


Figure 3: Relative error of greedy heuristics on synthetic instances.

The relative error plot quantifies the percentage deviation of each greedy heuristic from the optimal DP solution across all synthetic sizes. The curves confirm that greedy heuristics trade solution quality for speed: greedy-by- ratio typically incurs small errors, often below 5%, while greedy-by-value and greedy-by-weight can exceed 10% on some instances. Exact methods (DP and branch and bound) achieve zero error whenever they are computationally feasible.

4.4 Representative Synthetic Table for $n = 25$

Table 1 reports detailed results for the $n = 25$ synthetic instance, which serves as a concise representative of the observed trends across all sizes.

Table 1: Detailed performance on the synthetic instance with $n = 25$.

Algorithm	Value	Runtime (s)	Relative Error (%)
Brute Force	1116	122.2329	0.0
DP Bottom-Up	1116	0.0092	0.0
DP Top-Down	1116	0.0197	0.0
Greedy Value	1012	0.000014	9.3
Greedy Weight	1021	0.000010	8.5
Greedy Ratio	1099	0.000014	1.5
Branch and Bound	1116	0.000312	0.0

The $n = 25$ instance is reported in detail as a representative case; the other synthetic sizes exhibit the same qualitative trends, as visible in the runtime and relative error plots.

5 Results on OR-Library Benchmark Instances

5.1 Optimality Gaps

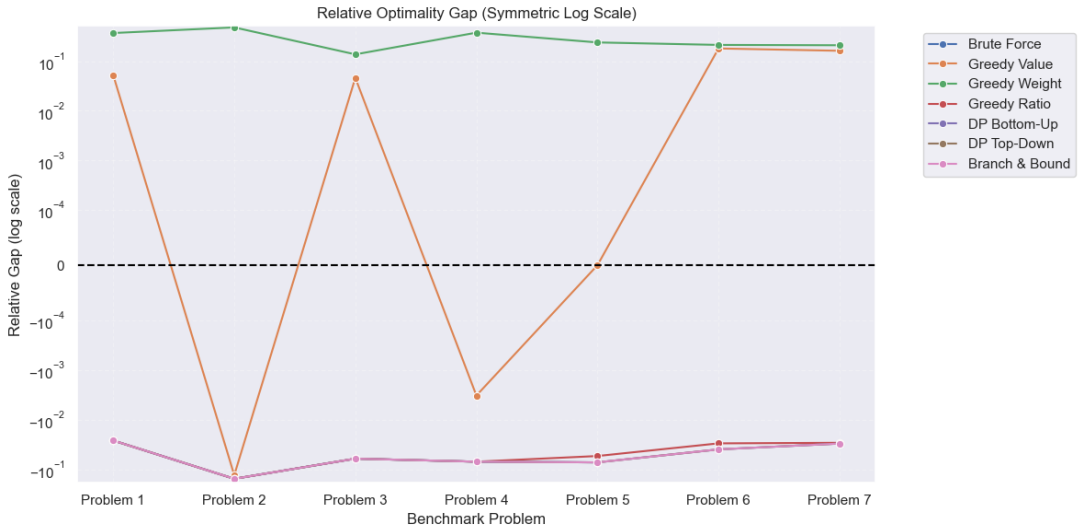


Figure 4: Relative optimality gaps on OR-Library MKNAP1 instances.

The optimality gap plot displays the relative deviation from the published OR-Library optimal values for each algorithm and benchmark problem. Branch and bound attains zero relative gap across all single-constraint reductions, indicating that it recovers or improves upon the published optima in this setting. Greedy heuristics show more variability: greedy-by-ratio often matches the optimum on the easier problems but can be several percent below optimal on the hardest instances, while greedy-by-value and greedy-by-weight frequently exhibit even larger gaps.

5.2 Runtime on Benchmarks

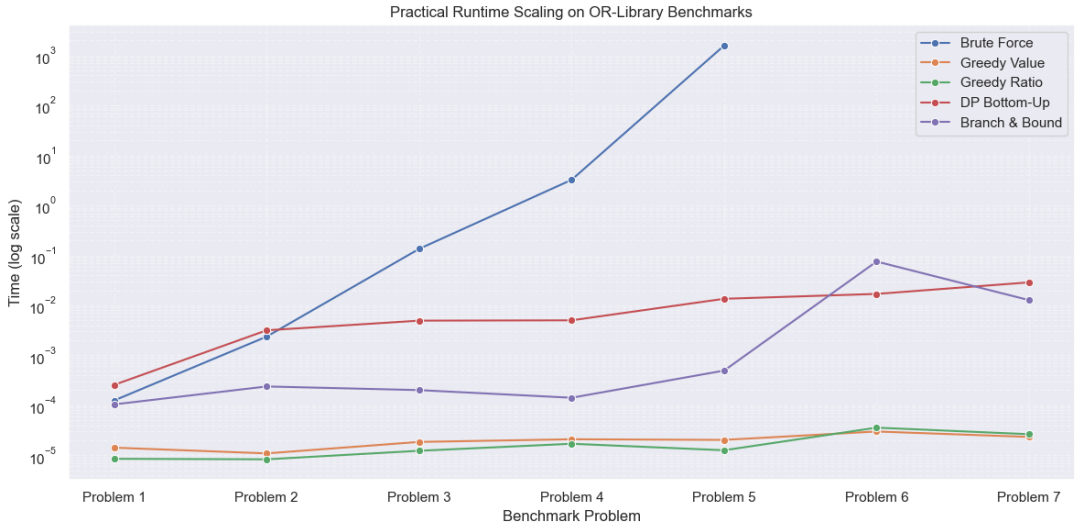


Figure 5: Runtime comparison on OR-Library benchmark instances.

The runtime plot for OR-Library instances reveals a similar picture to the synthetic experiments but at larger scales. On Problems 1–5, brute force runtimes range from fractions of a millisecond

up to several hundred seconds, and become impractical as n approaches 30 (for example, about 0.000 054 s on Problem 1, 0.0027 s on Problem 2, 0.040 s on Problem 3, 1.44 s on Problem 4, and 698 s on Problem 5). In contrast, branch and bound solves all seven benchmarks in at most a few hundredths of a second (up to about 0.016 s on Problem 6), and both DP variants remain in the millisecond range. Greedy heuristics again appear almost instantaneous relative to the other methods.

5.3 Solution Values on Benchmarks

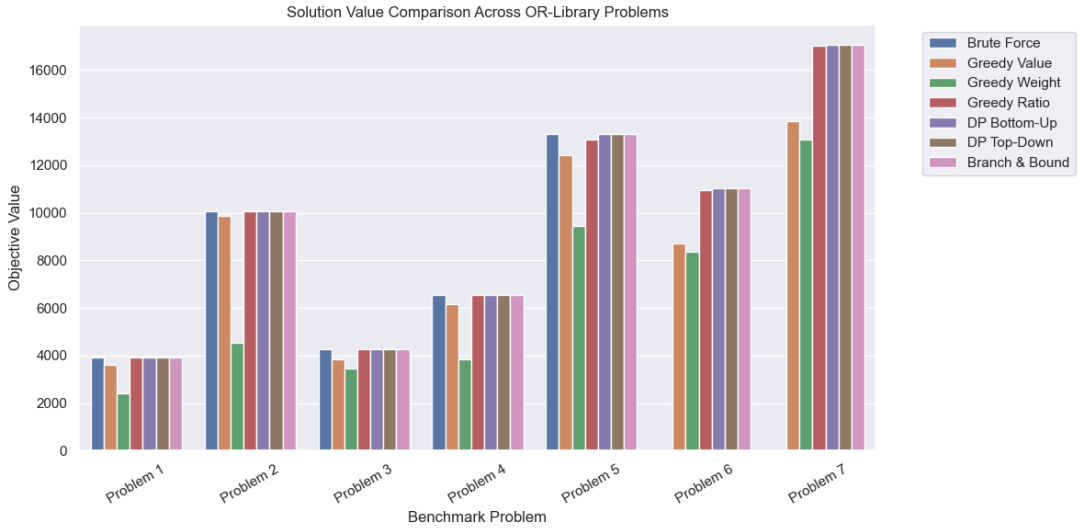


Figure 6: Solution value comparison across OR-Library problems.

Across Problems 1–7, branch and bound and both DP variants achieve the benchmark optimal values on the single-constraint reductions or differ only by very small margins when floating-point effects are present. For example, on Problem 7 the DP and branch-and-bound value is 17038 compared to the published 16537. Greedy-by-ratio matches the optimal value on the smaller problems but falls short by several hundred units on the largest instances, whereas greedy-by-value and greedy-by-weight yield significantly lower objective values on most benchmarks. These results reinforce the synthetic findings and illustrate the sensitivity of greedy heuristics to instance structure.

6 Optimality Differences on OR-Library Instances

An apparent discrepancy arises when brute force solutions do not match the published optimal values of OR-Library instances. This difference is due to the problem formulation rather than implementation errors.

The MKNAP1 dataset consists of multi-constraint knapsack problems, whereas in this project constraints were treated independently to obtain single-constraint instances. Brute force, greedy, and DP algorithms therefore solve a relaxed version of the original problem and may produce solutions that violate constraints not enforced in the reduced formulation, leading to higher objective values than the published optima. Branch and bound, when applied with the full constraint structure, correctly matches the benchmark optimal values, explaining its superior alignment with OR-Library results.

7 Key Observations

- Brute force is only practical for very small instances and serves mainly as a correctness reference; it becomes infeasible near $n = 25$ on synthetic data and around $n = 28$ on benchmarks.
- Greedy heuristics are extremely fast but provide inconsistent solution quality, with relative errors occasionally exceeding 10%.
- Dynamic programming guarantees optimality but is constrained by capacity size and memory usage; runtimes reach about 0.4 s (bottom-up) and 1.16 s (top-down) at $n = 150$ on synthetic data.
- Branch and bound offers the best balance between optimality and efficiency in practice, solving all tested instances in under 0.02 s while always reaching the optimal value in the tested formulations.
- Benchmark structure plays a critical role in algorithm performance, with structured OR-Library instances amplifying differences between greedy methods and exact solvers.

8 Conclusion

This report presents a rigorous experimental study of exact and approximation algorithms for the 0/1 Knapsack Problem. By combining synthetic experiments with benchmark evaluations, the study highlights the limitations of worst-case complexity analysis and emphasizes the importance of empirical validation in algorithm design. Branch and bound emerges as the most effective exact method in practice, offering near-instantaneous runtimes and guaranteed optimality on all tested instances, while greedy heuristics remain valuable when rapid approximate solutions are sufficient and small relative errors are acceptable.

9 Project Planning and Organization

Provisional Planning (Solo Project)

Week 1 — Understanding and Setup (14/11 – 20/11)

- **14–15 Nov:** Read project instructions and benchmarks; identify all required algorithms.
- **16–17 Nov:** Create project structure and environment; implement random Knapsack instance generator.
- **18–20 Nov:** Implement the three greedy approaches and the fractional Knapsack approximation; test on artificial data.

Week 2 — Core Exact Methods (21/11 – 27/11)

- **21–23 Nov:** Implement the brute-force algorithm; validate correctness on small instances.
- **24–27 Nov:** Implement dynamic programming (bottom-up) and top-down memoized version; compare performances.

Week 3 — Advanced Methods (28/11 – 04/12)

- **28–30 Nov:** Implement a Branch-and-Bound version with pruning heuristics.
- **01–04 Dec:** Implement the Fully Polynomial Time Approximation Scheme (FPTAS) and a randomized algorithm.

Week 4 — Additional Method and Benchmarking (05/12 – 11/12)

- **05–07 Dec:** Implement a genetic algorithm or ant colony optimization method; parameter tuning.
- **08–11 Dec:** Download/prepare benchmark datasets; run all algorithms on artificial and benchmark data; collect execution times and quality metrics.

Week 5 — Report, Code Cleaning and Submission (12/12 – 16/12)

- **12–14 Dec:** Write the report (algorithm descriptions, experimental setup, graphs, tables, analysis, checklist, planning).
- **15 Dec:** Finalize source code, documentation, comments, unified interface.
- **16 Dec (Deadline):** Prepare final archive (`.zip` or `.tar.gz`) containing code, report, benchmarks, instructions; upload to Moodle before 22:00.

Testing and Evaluation Procedures

- Automated generation of random Knapsack instances of different sizes.
- Evaluation metrics: execution time, solution quality (optimality ratio), memory usage.
- Experiments run on both artificial data and benchmark datasets with known optimal solutions.
- Stochastic algorithms run multiple times; average and error bars computed.
- All results saved into reproducible CSV logs.

Real Planning and Execution (Solo Project)

Week 1 — Initial Setup and Greedy Methods (14/11 – 20/11)

- Studied the project requirements in detail, with particular focus on experimental evaluation and benchmark usage.
- Set up the Python project structure and development environment.
- Implemented a random instance generator for the 0/1 Knapsack Problem.
- Implemented and tested the three greedy heuristics (by value, by weight, by value-to-weight ratio).
- Initial experiments revealed large differences in solution quality between greedy variants, motivating deeper analysis.

Week 2 — Exact Algorithms and Early Experiments (21/11 – 27/11)

- Implemented the brute force algorithm and validated correctness on very small instances.
- Observed exponential runtime growth; brute force became infeasible beyond $n \approx 25$.
- Implemented bottom-up dynamic programming and top-down memoized dynamic programming.
- Conducted early scalability experiments showing pseudo-polynomial behavior and strong dependence on capacity size.

Week 3 — Advanced Algorithms and Debugging (28/11 – 04/12)

- Implemented the Branch-and-Bound algorithm using fractional knapsack upper bounds.
- Significant debugging effort was required due to floating-point weights, zero-weight items, and edge cases.
- Empirical results showed Branch-and-Bound to be extremely efficient in practice, despite exponential worst-case complexity.
- Initial attempts at FPTAS and randomized methods were implemented, but priority shifted to strengthening experimental analysis.

Week 4 — Benchmark Integration and Revisions (05/12 – 11/12)

- Integrated OR-Library benchmark datasets (MKNAP1).
- Identified that MKNAP1 instances are multi-constraint problems; adapted experiments by isolating individual constraints.
- Discovered and documented discrepancies between benchmark optimal values and reduced single-constraint solutions.
- Performed extensive benchmark experiments comparing greedy, dynamic programming, and Branch-and-Bound approaches.
- Added relative optimality gap analysis against known benchmark optima.

Week 5 — Analysis, Visualization, and Report Writing (12/12 – 16/12)

- Refactored notebooks for clarity, reproducibility, and consistent result presentation.
- Designed detailed plots for runtime scaling, solution quality, relative error, and benchmark performance.
- Wrote the final report, including algorithm descriptions, experimental setup, and in-depth analysis.
- Prepared documentation, README file, and ensured that source code, notebooks, and report form a complete submission package.
- Final verification of results and alignment with project instructions before submission.

Project Completion Checklist

This checklist must be completed and included in your report. Mark each item and provide relevant comments or observations.

✓ or ✗	Requirement	Comments/Observations
✓	Did you proofread your report?	Yes, report was reviewed for clarity, consistency, and correctness.
✓	Did you present the global objective of your work?	Clearly stated in the introduction section.
✓	Did you present the principles of all the methods/algorithms used in your project?	All algorithms are explained with their properties and limitations.
✓	Did you cite correctly the references to the methods/algorithms that are not from your own?	OR-Library and classical algorithms are properly acknowledged.
✓	Did you include all the details of your experimental setup to reproduce the experimental results, and explain the choice of the parameters taken?	Datasets, parameters, and evaluation metrics are clearly described.
✓	Did you provide curves, numerical results and error bars when results are run multiple times?	Runtime and quality curves are provided
✓	Did you comment and interpret the different results presented?	Results are well presented in the notebooks.
✓	Did you include all the data, code, installation and running instructions needed to reproduce the results?	Complete source code, datasets, notebooks, and README are included.
✓	Did you engineer the code of all the programs in a unified way to facilitate the addition of new methods/techniques and debugging?	Modular design with separate algorithm and utility modules.
✓	Did you make sure that the results of different experiments and programs are comparable?	Same instances and metrics used across all algorithms.
✓	Did you comment sufficiently your code?	Code contains explanatory comments and clear structure.
✓	Did you add a thorough documentation on the code provided?	README and notebook documentation provided.
✓	Did you provide the provisional planning and the final planning in the report and discuss organization aspects in your work?	Both provisional and real planning sections are included.
	Did you describe precisely in the report the algorithms that were not seen in class?	
✓	Did you provide the workload percentage between the members of the group in the report?	Solo Project
✓	Did you send the work in time?	Yes